

数据结构与算法 A
common-14 秋-期末

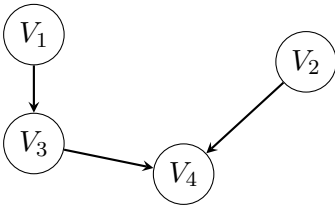
一、填空 (27 分)

1. 当各边上的权值 _____ 时，BFS 算法可用来解决单源最短路径问题。

- (A) 均相等
- (B) 均互不相等
- (C) 不一定相等

解答：
选 A。BFS 的层次扩展保证先到达的顶点经过的边数最少，因此它求的是”边数最少”的路径。若所有边权值均相等，则路径权值正比于边数，BFS 得到的就是单源最短路径；若边权不同，即使边数少也未必权值小，应改用 Dijkstra 等带权最短路算法。

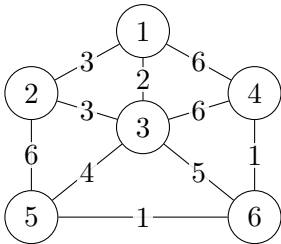
2. 有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
- (2) 添加一条弧 _____ 之后，则仅有唯一的拓扑序列。

解答：
拓扑序列共有 V_1, V_2, V_3, V_4 , V_1, V_3, V_2, V_4 , V_2, V_1, V_3, V_4 三个。添加弧 $V_2 \rightarrow V_1$ 后拓扑序列唯一为 V_2, V_1, V_3, V_4 ；添加弧 $V_3 \rightarrow V_2$ 也可得到唯一序列 V_1, V_3, V_2, V_4 。

3. 设有如下所示无向图 G ，用 Prim 算法构造最小生成树所走过的边



的边集。

解答：
从顶点 1 开始执行 Prim 算法，初始生成树顶点集为 $\{1\}$ ，每一步都在”已入树顶点”和”未入树顶点”之间选权值最小的边。按图中权值逐步扩展，可取边集
$$\{(1,3), (1,2), (3,5), (5,6), (6,4)\}.$$

若同权边取法不同，第二条边中的 $(1,2)$ 可由 $(2,3)$ 替代；两种结果的总权值相同，都满足最小生成树的割性质。阅卷时关键看每一步是否只从当前割边中选最小边，而不是先全局排序后随意取边。

4. 如果只想得到 1000 个元素组成的序列中第 5 个最小元素之前的部分排序的序列，用 _____ 方法最快。

- (A) 起泡排序 (C) 堆排序
(B) 快速排序 (D) 简单选择排序

解答：

选 C。目标不是把 1000 个元素全部排好，而是尽快确定前 5 个最小元素。可维护一个大小为 5 的最大堆，扫描其余元素时只在新元素小于堆顶时替换并调整，最终堆中即为最小的 5 个元素；若还要求这 5 个元素内部有序，再对这 5 个元素排序即可，代价远小于完整排序。

5. 若要求尽可能快地对序列进行稳定的排序，则应选 _____。

- (A) 快速排序 (C) 冒泡排序
(B) 归并排序 (D) 堆排序

解答：

选 B。快速排序和堆排序通常不稳定；冒泡排序稳定但平均时间为 $O(n^2)$ 。归并排序在比较排序中能稳定地达到 $O(n \log n)$ ，并且稳定性来自合并两个有序段时相等元素按原先相对次序输出。

6. 在文件局部有序或文件长度较小的情况下，最佳的排序方法是 _____。

- (A) 直接插入排序 (C) 直接选择排序
(B) 冒泡排序 (D) 堆排序

解答：

选 A。直接插入排序每趟只需把当前元素插入前面已有序部分；当文件已接近有序时，元素移动距离短，比较和移动次数接近线性。文件长度较小时，插入排序常数小、实现简单，也优于需要递归或建堆等额外开销的方法。

7. 设输入的关键字满足 $K_1 > K_2 > \dots > K_n$ ，缓冲区大小为 m ，用置换选择排序方法可产生 _____ 个初始归并段。

解答：

答案按题目口径为 $n/m + 1$ 。置换选择排序中，缓冲区大小为 m ；当输入关键字严格递减时，新读入元素总小于刚输出的元素，不能进入当前归并段，只能被冻结到下一段。因此每个初始归并段最多输出缓冲区中原有的 m 个元素，段数按整段估计为 n/m ，若最后还有不足一段的冻结元素则再形成一段，更严谨地可写作 $\lceil n/m \rceil$ 。

8. 设有 13 个初始归并段，其长度分别为 28, 16, 37, 42, 5, 9, 13, 14, 20, 17, 30, 12, 18。试计算最佳 4 路归并树的带权路径长度 $WPL =$ _____。

解答：

最佳 4 路归并树 WPL 为 480。4 路 Huffman 合并要求叶子数满足 $(n - 1) \bmod (4 - 1) = 0$ ；原有 13 个归并段已满足该条件，因此不需要补 0 长度虚段。每次取四个最短段合并：

$$5, 9, 12, 13 \rightarrow 39, \quad 14, 16, 17, 18 \rightarrow 65, \quad 20, 28, 30, 37 \rightarrow 115, \quad 39, 42, 65, 115 \rightarrow 261.$$

带权外部路径长度等于各次合并长度之和，故 $WPL = 39 + 65 + 115 + 261 = 480$ 。

9. 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908，则对其进行折半查找时，等概率下搜索成功的平均查找长度和不成功的平均查找长度分别为 _____ 和 _____。

解答：

折半查找成功 ASL 为 45/14；不成功 ASL 为 59/15。计算时把折半查找过程画成判定树：14 个元素的成功结点比较次数分别等于其在判定树中的层数，求和得 45；失败查找对应 15 个外部空指针位置，其比较次数求和为 59。等概率时分别除以成功情形数 14 和失败间隙数 15。

10. 设散列表长度为 14(地址下标为 0...13)，哈希函数为 $H(key)=key\%11$ ，表中已有数据的关键字为 15,38,61,84 共四个，现要将关键字为 49 的结点加入表中，用二次探查法解决冲突，则放入的位置是 _____。

- (A) 8 (C) 5
(B) 3 (D) 9

解答：

选 D。已有关键字 15,38,61,84 的散列地址均为 4,5,6,7 中的冲突链位置； $H(49) = 49 \bmod 11 = 5$ 。二次探查按 $5 \pm 1^2, 5 \pm 2^2, \dots$ 检查，位置 5 已占，6 已占，再检查 $5 + 2^2 = 9$ ，位置 9 空，故插入到 9。

11. 在一棵含有 1023 个关键字的 4 阶 B 树中进行查找 (不读取数据主文件)，至多读盘 _____ 次。

- (A) 7 (C) 10
(B) 8 (D) 9

解答：

选 C，即至多读盘 10 次；该题不同教材对“阶”和读盘层数的计数口径可能不同，这里按本卷给出的选项口径作答。4 阶 B 树中每个结点最多有 3 个关键字、4 个孩子；在最稀疏情形下，除根外每个内部结点至少有 2 个孩子、至少 1 个关键字。估计最坏查找路径时，应按最小分支数逐层展开，得到容纳 1023 个关键字所需的最大层数；查找每下一层通常对应一次读盘，所以最大读盘次数等于查找路径上访问的结点数。

12. 在一棵阶为 k 的红黑树中，内部结点最少有 $2k-1$ 个；从根到叶的简单路径长度的范围为 $[k,2k]$ 。

解答：

该说法按“黑高度为 k”理解时成立：红黑树中从某结点到任一外部叶子的黑结点数相同，且红结点不能有红孩子，因此黑高度为 k 时，内部结点数至少出现在全黑的最瘦情形，至少为 $2^k - 1$ ；任一路径上红结点数不超过黑结点数，所以实际高度至多约为 $2k$ 。若题面中的“阶为 k”是指普通高度，则“最少有 $2k - 1$ 个内部结点”并不一定成立，表述应改成关于黑高度的命题。

二、辨析与简答 (30 分)

13. (8 分) 将关键字序列 (7、8、30、11、18、9、14) 散列存储到散列表中。散列表是一个下标从 0 开始的一维数组，散列函数为： $H(key) = (key * 3) \bmod 7$ ，处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。(1) 请画出所构造的散列表。(2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

解答：

散列表长度 $L = \lceil 7/0.7 \rceil = 10$ 。 $H(key) = 3key \bmod 7$ ，线性探测结果如下：

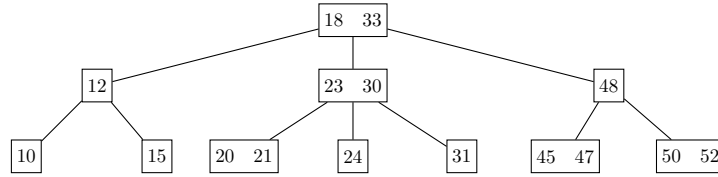
地址	0	1	2	3	4	5	6	7	8	9
关键字	7	14	-	8	-	11	30	18	9	-

成功查找比较次数分别为 1,1,1,1,3,3,2，故 $ASL_{succ} = 12/7$ 。不成功查找只需考虑初始地址 0 ~ 6：

初始地址	0	1	2	3	4	5	6
比较次数	3	2	1	2	1	5	4

故 $ASL_{unsucc} = 18/7$ 。

14. (6 分) 设有如下一棵 3 阶 B 树，请画出在其中插入关键码 19 后的 B 树，并给出



插入过程中的访外 (读写) 次数；在此基础上再删除关键码 48，请画出删除后的 B 树及删除过程中的访外 (读写) 次数。

解答：

插入 19：插入到叶结点 [20, 21] 后分裂，20 上升；父结点 [23, 30] 分裂，23 上升；根 [18, 33] 分裂，树高加 1。删除 48：用后继 50 替换 48，再从叶结点 [50, 52] 中删除 50，得到叶结点 [52]，无需再合并。

15. (6 分) 将下列字符序列 E A S Y Q U E S T I O N 依次插入到初始为空的红黑树 (RB-tree) 中，请画出最终得到的红黑树。(用圆圈表示红色结点，方框表示黑色结点，外部空叶结点可省略不画)。

解答：

依次插入 *E, A, S, Y, Q, U, E, S, T, I, O, N* 时，先按二叉搜索树规则找到插入位置，新结点着红色；若父结点为黑色则结束，若父结点和叔结点同为红色则将父、叔改黑、祖父改红并继续向上检查；若父红叔黑，则按 LL/LR/RL/RR 情形旋转并重新着色。重复关键字 *E, S* 可按“相等不插入”处理，也可在结点中计数，但不能破坏搜索树的中序有序性。最终红黑树必须满足：根为黑，所有红结点的孩子为黑，从根到每个外部空结点的黑高度相同；答案图只要满足这些约束且由合法插入序列得到即可。

16. (4 分) 利用广义表的 Head 和 Tail 运算，把原子 d 分别从下列广义表中分离出来， $L_1 = (((((a), b), d), e))$ ； $L_2 = (a, (b, ((d)), e))$ 。

解答：

对 $L_1 = (((((a), b), d), e))$ ，可用 $Head(Tail(Head(Head(L_1))))$ 分离出 *d*。对 $L_2 = (a, (b, ((d)), e))$ ，可用 $Head(Head(Head(Tail(Head(Tail(L_2)))))$ 分离出 *d*。

17. (6 分) 将关键码 1, 2, 3, ..., (2k-1) 依次插入到一棵初始为空的 AVL 树中，试证明结果是一棵高度为 k 的完全满二叉树。

解答：

用数学归纳法： $k = 1$ 时只有关键码 1，显然为高度 1 的满二叉树。设插入 $1, \dots, 2^k - 1$ 后为高度 k 的满 AVL 树；继续递增插入时，新结点总在最右链，触发的均为 RR 型旋转，旋转后左右子树高度相同，得到高度 $k + 1$ 的满二叉树。

三、算法填空 (18 分) 阅读和完成下面的代码 (每空可不限一条语句)。

18. (10 分) 以单向链表为存储结构，实现选择排序算法 (得到非递减序列)。

```
class Node:
    def __init__(self, key, next=None):
        self.key = key
        self.next = next

def sort_linked_list(head):
    i = head
    while i is not None:
        # 填空 1
        while j is not None:
```

```

        if ____:          # 填空 2
            # 填空 3
            # 填空 4
            # 填空 5
    return head

```

解答:

填空依次为: `j = i.next; i.key > j.key; i.key, j.key = j.key, i.key; j = j.next; i = i.next`。外层指针 `i` 依次确定当前应放置最小关键字的位置, 内层指针 `j` 从 `i` 的后继开始扫描未排序部分; 若发现 `j` 的关键字更小, 就交换两个结点中的关键字。内层结束后, `i` 位置保存的是剩余元素中的最小值, 再让 `i` 后移, 故这是链表上的简单选择排序写法。

19. (8 分) 下面是败者树在内部结点从右分支向上比赛的成员函数实现, 请将空缺部分补充完整。

```

def play(p, lc, rc, winner, loser):
    B[p] = loser(L, lc, rc)
    # 填空 1
    while p > 1 and p % 2 == 1:
        temp2 = winner(L, temp1, B[p // 2])
        # 填空 3
        temp1 = temp2
        p //= 2
    # 填空 4

```

解答:

填空依次为: `temp1 = winner(L, lc, rc); B[p // 2] = loser(L, temp1, B[p // 2]);` 最后 `B[p // 2] = temp1`。该段代码的思想是: 先在左右孩子 `lc, rc` 中取胜者作为 `temp1`, 然后沿着从被修改叶子到根的路径逐层重赛; 每个内部结点保存败者, 胜者继续向上。循环条件中的 `p % 2 == 1` 用来判断当前结点是否位于右分支, 最后把一路胜出的选手放到根附近相应位置。

20. (8 分) 下述算法实现在图 `G` 中计算从顶点 `i` 到顶点 `j` 之间长度为 `len` 的简单路径条数。图的 ADT 如下:

```

class Graph:
    def first_edge(self, vertex): ...
    def next_edge(self, edge): ...
    def is_edge(self, edge): ...
    def to_vertex(self, edge): ...

visited = [False] * MAXSIZE

def get_path_num_len(G, i, j, length):
    if ____:          # 填空 1
        return 1
    total = 0          # total 表示通过本结点的路径数
    visited[i] = True
    e = G.first_edge(i)
    while G.is_edge(e):
        v = G.to_vertex(e)
        if not visited[v]:
            # 填空 2
            e = G.next_edge(e)
    visited[i] = False  # 允许该结点出现在另一条路径中
    return total

```

解答：

填空为: `i == j and length == 0`; 以及 `total += get_path_num_len(G, v, j, length - 1)`。递归含义是“从当前顶点 i 出发, 还需要走 $length$ 条边到达 j 的简单路径条数”。当 $length = 0$ 时, 只有当前顶点已经是 j 才计为一条路径; 否则应枚举 i 的每个未访问邻接点 v , 标记后递归求从 v 到 j 、剩余长度为 $length - 1$ 的路径数, 并累加到 `total`。返回前必须恢复 `visited[i]`, 否则会影响其他分支的枚举。

四、算法设计与分析 (25 分) 请尽量按照试题中的要求来写高效率的可读算法。应该申明算法思想, 在代码中加以恰当的注释。

21. 伸展树是一种自平衡的 BST 树, 它可以通过旋转来实现树结构的动态调整。伸展树结点的数据结构定义如下:

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.father = None
        self.left = None
        self.right = None
```

其成员函数包括: 1) `Delete(x)`: 删除以 x 结点为根的子树; 2) `Splay(x, f)`: 将 x 旋转为 f 的子结点 (f 是 x 的祖先)。特殊地, 把 x 旋转到根结点即 `Splay(x, None)`。请运用上述给定的成员函数, 实现删除树 `rt` 中所有大于 u 小于 v 的结点 (假设 u, v 是 `Splay` 树中的结点, 且树中所有结点值不重复)。函数原型为: `delete_between(root, u, v)`。解答: 把 u 结点旋转到根; 把 v 旋转为 u 的右儿子; 删除 v 结点的左子树前面几句各 3 分, 最后一句 1 分。如果思路、注释各 1 分, 不写则扣。

```
def delete_between(root, u, v):
    Splay(u, None)
    Splay(v, u)
    Delete(v.left)
    v.left = None
    return u
```

解答：

先执行 `Splay(u, None)`, 使 u 成为整棵树的根。由于 $u < v$, 所有落在区间 (u, v) 中的关键码都位于根 u 的右子树内; 再执行 `Splay(v, u)`, 只在 u 的右子树中把 v 伸展为 u 的右孩子。此时 v 的左子树恰好包含所有大于 u 且小于 v 的结点, 而 v 的右子树包含大于 v 的结点。调用 `Delete(v.left)` 释放该左子树并令 `v.left = None`, 即可删除开区间内全部元素, 同时保留区间外元素的二叉搜索树次序。

数据结构与算法 A

common-15 秋-期末

一、选择题 (每题 2 分, 共 20 分)

1. 若要一个运行时间为 $100n^2$ 的算法在相同机器上快于另一个运行时间为 2^n 的算法, 则最小的 n 值应为 _____。

解答:

若按合理题意理解为比较 $100n^2$ 与 2^n , 最小 n 为 15。检验临界值: $n = 14$ 时 $100n^2 = 19600$, 而 $2^{14} = 16384$, 指数项还没有超过; $n = 15$ 时 $100n^2 = 22500$, 而 $2^{15} = 32768$, 首次超过。若严格按题面中的 $2n$ 理解, 则线性函数 $2n$ 不可能在正整数范围内超过 $100n^2$, 因此题目应有排版或 OCR 误差。

2. 某算法仅含顺序执行的语句 1 和语句 2, 语句 1 执行次数为 $20n^2 + 3n \log n$, 而语句 2 执行次数为 $2n^3$, 则该算法的时间复杂度为 _____。

解答:

时间复杂度为 $O(n^3)$ 。顺序执行的两段代码总执行次数相加为 $20n^2 + 3n \log n + 2n^3$ 。当 n 足够大时, 三次项增长最快, 低阶项和常数系数在大 O 记号中忽略, 因此总时间复杂度为 $O(n^3)$ 。

3. 下面术语中, _____ 与数据结构的存储结构无关。

(A) 顺序表 (B) 链表 (C) 队列 (D) 循环链表 (E) 堆

解答:

与存储结构无关的是 C、E。顺序表、链表、循环链表都直接描述元素在存储器中的组织方式; 队列描述队尾入队、队首出队的操作约束, 属于逻辑结构/抽象数据类型。堆强调完全二叉树上的偏序关系, 通常可用数组实现, 但概念本身不是某一种存储结构。

4. 在一个具有 n 个结点的有序单链表中插入一个新结点并仍保持其有序的时间复杂度为 _____。

(A) $O(n \log_2 n)$ (C) $O(n)$
(B) $O(1)$ (D) $O(n^2)$

解答:

选 C, 时间复杂度为 $O(n)$ 。有序单链表在已知前驱结点时插入只需 $O(1)$ 修改指针, 但为了保持有序, 必须从表头顺链比较关键字直到找到插入位置。平均检查约一半结点, 最坏检查全部结点, 因此时间阶为 $O(n)$ 。

5. 若元素 a 、 b 、 c 、 d 、 e 、 f 依次进栈, 允许进栈、退栈操作交替进行, 但不允许连续三次进行退栈操作, 则不可能得到的出栈序列是 _____。

(A) dcebfa (C) bcaefd
(B) cbdaef (D) afedcb

解答:

选 D。若第一个出栈元素为 a , 必须先压入 a 后立即弹出。随后要输出 f, e, d, c, b , 需要继续压入 b, c, d, e, f , 再按栈的逆序连续弹出 $f, e, d, \dots$ 。其中 f, e, d 已经构成连续三次出栈, 违反题设限制, 所以不可能。

6. 表达式 $3 * 2^{(4 + 2 * 2 - 6 * 3)} - 5$ 的求值过程中, 当扫描到 6 时, 操作数栈和运算符栈为 _____, 其中 $^$ 为乘幂。

- (A) 3, 2, 4, 1, 1; $(\wedge + * -$
 (B) 3, 2, 8; $(\wedge -$
 (C) 3, 2, 4, 2, 2; $(\wedge (-$
 (D) 3, 2, 8; $(\wedge (-$

解答:

选 D。括号内扫描到 $4 + 2 * 2$ 时, 乘法先算出 $2 * 2 = 4$, 再与前面的 4 合成 8; 遇到减号后, 括号内当前等待的运算符为 $-$ 。外层的 $*$ 、乘幂和左括号尚未出栈, 所以操作数栈为 3, 2, 8, 运算符栈为 $(\wedge (-$ 。

7. 有 4 棵结点存储整数关键码的二叉树, 若它们的中序遍历序列分别为: A. 5, 3, 1, 2, 4; B. 1, 2, 3, 4, 5; C. 5, 4, 3, 2, 1; D. 1, 3, 5, 4, 2。请问其中可能是二叉搜索树的包括 _____。

解答:

按通常二叉搜索树定义, 左子树关键码小于根、右子树关键码大于根, 因此中序遍历应得到关键码的非递减序列。四个候选序列中只有 1, 2, 3, 4, 5 严格递增, 因此选 B; 其他序列都存在逆序对, 例如先出现较大关键码再出现较小关键码, 这会违反“中序遍历先访问左子树、再访问根、再访问右子树”的有序性, 所以不可能是二叉搜索树的中序遍历。

8. (4 分) 一棵 Huffman 树的带权外部路径长度定义为所有叶结点权值与其深度的乘积之和。设根结点的深度为 0, 对集合 {2, 3, 5, 7, 8} 构造二叉 Huffman 树, 则其最小带权外部路径长度为 _____。

解答:

Huffman 合并时每次取当前最小的两个权值。先合并 2 和 3 得到 5, 再合并两个 5 得到 10, 再合并 7 和 8 得到 15, 最后合并 10 和 15 得到根 25。Huffman 树的 WPL 等于每次合并产生的新权值之和, 因此

$$WPL = 5 + 10 + 15 + 25 = 55.$$

这比逐个画出每个叶子的编码长度更快捷, 但本质上等价于把每个叶权乘以其路径长度后求和。

9. 两个字符串相等的充分必要条件是 _____。(两字符串的长度相等且两字符串中对应位置的字符也相等)

解答:

两个字符串相等当且仅当长度相等且对应位置字符均相等。比较时通常先比较长度; 若长度不同, 即使较短串是较长串的完整前缀, 也不是同一字符串。长度相同时, 再从第一个字符开始逐位比较, 只要存在某个位置字符不同, 就可立即判定两个串不相等; 只有所有位置都相同, 才判定为相等。

10. 2-3 树是一种满足以下两个条件的特殊的树: (1) 每个内部结点有两个或三个子结点; (2) 所有叶结点到根的路径长度相同。如果一棵 2-3 树有 9 个叶结点, 那么它可能有 _____ 个非叶结点。

解答:

可能有 4 或 7 个非叶结点, 取决于题目采用的高度和叶层口径。若根的三个孩子都是内部结点, 并且这三个内部结点各有三个叶子, 则叶结点数为 $3 \times 3 = 9$, 非叶结点为根加三个孩子, 共 4 个。若允许在同一层继续把部分孩子细分为内部结点, 同时仍保持所有叶子在同一层, 则也可构造出 7 个非叶结点。作答时应先说明采用的 2-3 树定义: 每个内部结点有 2 或 3 个孩子, 且所有叶子在同一层; 再根据该定义数内部结点, 不能只凭叶子数直接唯一确定非叶结点数。

二、辨析与简答题 (共 36 分)

11. (6 分) 什么是循环队列? 循环队列的优点是什么? 如何判别它的空和满?

解答:

循环队列把一维数组看成首尾相接, 用取模运算让队尾到达数组末端后回到下标 0。它能克服普通顺序队列中“数组后端满但前端已有空位”的假溢出现象。若采用少用一个存储单元的约定, 队空为 $front = rear$; 入队前若 $(rear + 1) \bmod maxSize = front$, 则队满。也可额外维护元素个数, 此时队空为 $count = 0$, 队满为 $count = maxSize$ 。

12. (6 分) 在包含 n 个结点的单链表、双链表和单循环链表中, 若仅知道指针 p 指向某结点, 不知道表的头指针, 能否将结点 p 从相应的链表中删去? 若可以, 其时间复杂度各为多少?

解答:

单链表仅知 p 通常不能真正删除 p , 因为要修改前驱结点的 `next` 指针, 而前驱无法由 p 反向得到; 若 p 不是尾结点, 可用“把后继结点的数据复制到 p , 再删除后继结点”的折中办法, 但这改变的是结点内容而非原结点身份。双链表可由 p 的前驱/后继指针直接改链, 时间 $O(1)$ 。单循环链表虽然能从 p 出发绕一圈找到前驱, 但最坏要扫描整表, 时间 $O(n)$ 。

13. (8 分) 请分别计算模式 $p = \text{"aabaac"}$ 优化前和优化后的 `next` 数组, 并按照优化后的 `next` 数组针对目标 $t = \text{"aabaabaabaac"}$ 进行 KMP 快速模式匹配, 请画出匹配过程的示意图并计算匹配过程中的比较次数。

解答:

优化后 `next` 向量为 $[-1, -1, 1, -1, -1, 2]$ 。求解时先按普通 KMP 的最长相等真前后缀得到回退位置, 再检查若 $P[i]$ 与回退后的字符相同, 则继续沿 `next` 回退, 从而避免匹配时立即重复比较同一字符。用该向量匹配 $t = \text{"aabaabaabaac"}$ 时, 前两段 `aabaab` 基本顺利推进, 遇到末尾不匹配时按 `next` 跳转而不是回到主串下一位重新开始; 逐次记录主串字符比较, 可得比较次数为 14。

14. (8 分) 根据树的双标记层次遍历序列, 求其带度数的后根遍历序列。譬如, 已知一棵树的双标记层次遍历序列如下:

```
A : ltag: 0 , rtag 1 B : ltag: 0 , rtag 0 C : ltag: 0 , rtag 1 D : ltag: 1 , rtag 0 G :
ltag: 0 , rtag 1 E : ltag: 0 , rtag 1 H : ltag: 1 , rtag 1 F : ltag: 1 , rtag 0 I : ltag:
1 , rtag 1
```

则其带度数的后根遍历序列为: D0 H0 G1 B2 F0 I0 E2 C1 A2(注: 各个结点按照“结点名度数”的方式给出, 结点之间用空格分隔)。现某棵树的双标记层次遍历序列如下:

```
A : ltag: 0 , rtag 1 B : ltag: 0 , rtag 0 G : ltag: 1 , rtag 1 C : ltag: 0 , rtag 0 F :
ltag: 0 , rtag 1 D : ltag: 1 , rtag 0 E : ltag: 1 , rtag 1 H : ltag: 1 , rtag 0 I : ltag:
1 , rtag 1
```

请给出其带度数的后根遍历序列。

解答:

带度数后根遍历序列为 $D0\ E0\ C2\ H0\ I0\ F2\ B2\ G0\ A2$ 。求解时先由双标记层次序恢复父子关系: $ltag = 0$ 表示有左子, $rtag = 0$ 表示有右兄弟; 然后对树做后根遍历, 即先依次访问所有孩子子树, 最后访问当前结点, 并在结点名后写出孩子个数。

15. (8 分) 使用重量权衡合并规则 (权重合并规则) 与路径压缩, 对下列从 0 到 15 之间的数的等价对进行归并。在初始情况下, 集合中的每个元素分别在独立的等价类中。当两棵树规模同样大时, 使结点数值较大的根结点作为值较小的根结点的子结点。

(0, 2) (1, 2) (3, 4) (3, 1) (3, 5) (9, 11) (12, 14) (3, 9) (4, 14) (6, 7) (8, 10) (8, 7) (7, 0) (10, 15) (10, 13)

请填写下面表格的空白部分树的父指针表示法的数组表示。也就是所有等价对都被处理之后, 所得父结点的下标值 (没有父结点则填“-1”)。

父结点下标

结点值

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

结点的下标 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

解答:

父指针数组为 $[-1, 0, 0, 0, 0, 0, 0, 6, 0, 0, 0, 9, 0, 0, 12, 0]$ 。合并时始终把规模小的树挂到规模大的树根下；规模相同则根值较大的挂到根值较小的树下。路径压缩只在查找根的过程中改变沿途结点父指针，因此最终多数结点直接指向代表元 0，但 7, 11, 14 等结点因最后一次压缩路径不同，仍分别指向 6, 9, 12。

三、算法填空题 (每题 12 分, 共 24 分)

16. (每空 3 分, 共 12 分) 下面的算法利用一个栈将一个给定的序列从小到大排序。长度为 n 的序列由 $1, 2, \dots, n$ 这 n 个连续的正整数组成。请补全下面的代码段, 使其可以判断给定的序列是否可以利用一个栈进行排序, 如果可以, 输出相应的操作。

例如: 序列 4 3 1 2, 此序列可以排序, 输出: push push push pop push pop pop pop。

```
class Stack:
    def clear(self): ...
    def push(self, item): ...
    def pop(self): ...
    def top(self): ...
    def is_empty(self): ...
    def is_full(self): ...

s = Stack()                # s 为栈
sequence = [...]           # 待排序序列
n = len(sequence)

def is_legal():             # 判断序列是否可以排序
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if ____:    # 填空 1
                    return False
    return True

def transform():            # 输出转换操作
    cur, i = 1, 0
    while ____:             # 填空 2
        while s.is_empty() or ____: # 填空 3
            ____            # 填空 4
            print("push", end=" ")
            if cur == s.top():
                break
        s.pop()
        print("pop", end=" ")
        cur += 1
```

解答:

填 空 依 次 为: $sequence[k] < sequence[i] < sequence[j]$; $cur \leq n$; $cur < s.top()$; $s.push(sequence[i])$; $i += 1$ 。第一空是在三重下标中判断是否出现严格递增的三元组; 后面几空对应栈模拟扫描过程: cur 表示当前希望弹出的目标值, 只有当还未越界且栈顶正好满足输出条件时才弹出, 否则继续把输入序列元素入栈。这样既保证每个元素只入栈一次, 也能检测给定序列是否可由合法栈操作产生。

17. (每空 3 分, 分析 3 分, 共 12 分) 以下算法用来判断两棵树是否相同, 试补全下面的代码段, 并分析算法的运行时间代价。

```
def is_equal(root1, root2):
    while root1 is not None and root2 is not None:
        if root1.value() != root2.value():
            return _____ # 填空 1
        if _____: # 填空 2
            return False
        root1 = root1.right_sibling()
        root2 = root2.right_sibling()
    if _____: # 填空 3
        return True
    return False
```

解答：

填空依次为: False; not is_equal(root1.left_most_child(), root2.left_most_child()); root1 is None and root2 is None。递归比较两棵树是否相等时，首先处理一空一非空的情形，应直接返回 False；若两个根都为空，则说明对应子树同时结束，应返回 True。非空时先比较根信息，再递归比较最左孩子子树和右兄弟链；任一部分不相等就返回 False。每个结点只被访问常数次数，故时间复杂度为 $O(n)$ ，递归栈深度为树高。

四、分析证明题 (共 20 分)

18. (5 分) 请证明非空满 K 叉树的叶结点数目为 $(K - 1) * n + 1$ ，其中 n 为分支结点数。

解答：

用归纳法证明。 $n = 0$ 时没有分支结点，树只含一个根结点且它也是叶结点，叶数为 $1 = (K - 1) \cdot 0 + 1$ 。假设有 n 个分支结点时叶数为 $(K - 1)n + 1$ 。再增加一个分支结点，等价于把原来的某个叶结点扩展成有 K 个孩子的内部结点：原叶减少 1 个，新叶增加 K 个，叶数净增 $K - 1$ 。因此叶数变为 $(K - 1)n + 1 + (K - 1) = (K - 1)(n + 1) + 1$ 。由归纳法，命题对所有 n 成立。

19. (15 分) 有 n 个数 $K_1, K_2, \dots, K_n$ 顺序存储在数组中，形成一棵 n 个结点的完全二叉树。现在要按如下方式建成一个最小值堆：从左至右扫描整个数组，将第 i 个元素 K_i ($i = 1, 2, \dots, n$) 与其父结点 $K_{\lfloor i/2 \rfloor}$ 比较，如果 $K_i \geq K_{\lfloor i/2 \rfloor}$ ，则不再进行操作；若 $K_i < K_{\lfloor i/2 \rfloor}$ ，将 2 个结点对换，再将 $K_{\lfloor i/2 \rfloor}$ 与 $K_{\lfloor i/4 \rfloor}$ 比较，以此类推，直到比较到根结点 K_1 。

- (1) 按照这种建堆方式，最坏情况下建堆的时间复杂度是多少？
- (2) 教材上使用的“筛选法”建堆 (Sift down)，最坏情况下建堆的时间复杂度是多少？
- (3) 上述 2 种建堆法有类似之处，请问它们的时间复杂度是否一样，若不同，则请说明导致它们时间复杂度不同的原因。

解答：

本题从左到右逐个上滤建堆，最坏情况下第 i 层结点上滤 i 次，总复杂度 $O(n \log n)$ ；筛选法自底向上下滤，越低层结点移动距离越短，总复杂度 $O(n)$ 。二者不同的根本原因是大量下层结点在上滤法中代价高，而在筛选法中代价低。

数据结构与算法 A

common-16 秋-期中

一、选择题 (每题 2 分, 共 20 分)

1. 下面函数的时间复杂度是_____。

```
def work(n, m):
    total = 0
    i = 1
    while i <= m:
        j = 1
        while j <= n:
            total += i * j
            j *= 2
        i *= 2
    return total
```

解答:

时间复杂度为 $O(\log m \log n)$ 。外层循环变量每次乘 2, 从常数增长到超过 m , 执行次数为 $\lfloor \log_2 m \rfloor + 1$ 量级; 对外层的每一次迭代, 内层循环变量也每次乘 2, 执行 $\lfloor \log_2 n \rfloor + 1$ 次。两层循环嵌套相乘, 低阶常数忽略后得到 $O(\log m \log n)$ 。

2. 下面关于线性表的叙述中, 正确的是哪些?

- (A) 采用顺序存储的线性表, 必须占用一片连续的存储单元。
- (B) 采用顺序存储的线性表, 便于进行插入和删除操作。
- (C) 采用链接存储的线性表, 不必占用一片连续的存储单元。
- (D) 采用链接存储的线性表, 便于插入和删除操作。
- (E) 采用链接存储的线性表, 插入第 i 个元素的时间与 i 的数值成正比。
- (F) 采用链接存储的线性表, 在已知的结点 p 后插入一个新节点的时间复杂度与结点 p 的位置有关。

解答:

正确项为 A、C、D、E。顺序存储用一段连续地址保存线性表元素, 因而能随机访问但插入删除可能移动大量元素; 链式存储的结点可分散在内存中, 通过指针维持逻辑次序, 不要求物理连续, 并且已知插入/删除位置的前驱时改指针即可。若题目只给“第 i 个位置”而未给前驱指针, 链表仍需从表头定位到第 $i-1$ 个结点, 时间与 i 有关, 所以不能简单说任意位置插入都为常数时间。

3. 若元素 a、b、c、d 依次进栈, 则可能的出栈序列有_____种; 若有元素 a、b、c、d、e 依次进栈, 则可能的出栈序列有_____种。

解答:

依次进栈且可任意合法出栈的序列数为 Catalan 数

$$C_n = \frac{1}{n+1} \binom{2n}{n}.$$

其原因是合法出栈过程可看成由 n 次入栈和 n 次出栈组成的合法括号序列: 任意前缀中出栈次数不能超过入栈次数。代入 $n=4$ 得 $C_4 = \frac{1}{5} \binom{8}{4} = 14$; 代入 $n=5$ 得 $C_5 = \frac{1}{6} \binom{10}{5} = 42$ 。

4. 一个字符串中_____称为该串的子串。ab cab 的不相等真子串个数为_____。

解答：

串中任意连续的字符序列称为子串。 $abcbab$ 的非空子串按长度统计：长度 1 有 a, b, c 共 3 个不同子串；长度 2 有 ab, bc, ca 共 3 个；长度 3 有 abc, bca, cab 共 3 个；长度 4 有 $abca, bcab$ 共 2 个；长度 5 是原串本身，真子串不计。因此不相等非空真子串共 $3 + 3 + 3 + 2 = 11$ 个。

5. 一个具有 n 个结点的二叉树的叶结点的个数最多为_____。

解答：

叶结点个数最多为 $\lceil n/2 \rceil$ 。要使叶子尽可能多，应让除叶子以外的结点尽量都有两个孩子，形成尽可能“满”的二叉树。若度为 0, 1, 2 的结点数分别为 n_0, n_1, n_2 ，二叉树满足 $n_0 = n_2 + 1$ ，且 $n = n_0 + n_1 + n_2$ 。在固定 n 时令 n_1 尽量小即可最大化 n_0 ：当 n 为奇数时 $n_1 = 0$ ， $n_0 = (n+1)/2$ ；当 n 为偶数时 $n_1 = 1$ ， $n_0 = n/2$ 。合并写作 $\lceil n/2 \rceil$ 。

6. 一个有 5 层结点的完全 3 叉树。按前序遍历周游给结点从 1 开始编号，则第 100 号结点的父结点的编码为_____。(注释：独根树为 1 层的)

解答：

完全三叉树 5 层均满时，根的三个子树各含 $1 + 3 + 9 + 27 = 40$ 个结点。先根编号中，根为 1，第一个子树编号为 2 到 41，第二个子树为 42 到 81，第三个子树为 82 到 121。第 100 号结点在第三个子树内，相对第三个子树根的先根编号为 $100 - 81 = 19$ ；在该子树中定位可得其父结点相对编号为 16，所以全树父结点编号为 $81 + 16 = 97$ 。

7. 将序列 {12, 70, 33, 65, 24, 56, 48, 92, 86, 33} 按建堆方法调整为小值堆，调整后的序列为_____。

解答：

建成小值堆后的层序序列可为 {12, 24, 33, 65, 33, 56, 48, 92, 86, 70}。检验时按数组下标看父子关系：12 不大于 24, 33；24 不大于 65, 33；33 不大于 56, 48；65 不大于 92, 86；33 不大于 70。因此每个父结点都不大于其孩子，满足小值堆条件。若采用自底向上筛选建堆，等值 33 的相对位置可能不同，但只要层序数组满足上述堆序性即可。

8. 假设用于通信的电文仅由 8 个字符 {a,b,c,d,e,f,g,h} 组成，字符在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10，对这些字符进行哈夫曼编码的外部路径长度为_____。

解答：

Huffman 外部路径长度为 2.61。合并权值依次为 $0.02+0.03 = 0.05$, $0.05+0.06 = 0.11$, $0.07+0.10 = 0.17$, $0.11+0.17 = 0.28$, $0.19+0.21 = 0.40$, $0.28+0.32 = 0.60$, $0.40+0.60 = 1.00$ ，故 WPL 为这些合并权值之和 2.61。

9. 将森林 F 转换为对应的二叉树 T，F 中的叶结点的个数为_____。

- (A) T 中叶结点的个数
- (B) T 中度为 1 的结点个数
- (C) T 中左子指针为空的结点个数
- (D) T 中右子指针为空的结点个数

解答：

选 C。森林转换为孩子兄弟二叉树时，结点的第一个孩子变为左孩子，右兄弟变为右孩子。因此森林中的叶结点正是“没有第一个孩子”的结点，在对应二叉树中表现为左子指针为空；右指针是否为空只反映它有没有右兄弟。

10. 一棵树 T 有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的结点个数为_____。

解答：

设总结点数为 N 。树中边数为 $N-1$ ，也等于所有结点度数之和。已知度数总和为 $20 \cdot 4 + 10 \cdot 3 + 1 \cdot 2 + 10 \cdot 1 = 122$ ，因此 $N-1 = 122$ ，得到 $N = 123$ 。叶结点数不必单独求出，也已包含在总数关系中。

二、辨析与简答 (共 32 分)

11. (6 分) 请谈谈循环队列和链表的优缺点。当遇到如下情况时，应当使用哪一种数据结构？

情况：你要维护食堂鸡腿饭的排队队伍，队伍有两种操作：1. 排在队首的人出来打鸡腿饭后离开；2. 一个新来的人排在队尾。你知道食堂不太大，所以队伍的最大长度你可以估计出来。

解答：

循环队列：顺序存储，空间连续，队头删除与队尾插入均为 $O(1)$ ，但容量需预先确定；链表：容量可动态增长，插删方便，但每个结点需额外指针空间且局部性较差。本题队伍最大长度可估计，且只有队头出队、队尾入队两个操作，宜用循环队列。

12. (4 分) 如何找出具有相同的中序序列和后序序列的所有二叉树？

解答：

若二叉树结点关键码互异，则中序序列与后序序列唯一确定一棵二叉树：后序末元素为根，在中序中分割左右子树，再递归构造。若关键码允许重复，则每次根值在中序中可能有多个位置，需要枚举所有可行分割并递归组合。

13. (4 分) 从空二叉树开始，将关键码 {18,73,10,5,68,99,27,41,51,32,25} 严格按照 BST(二叉搜索树) 的插入算法逐个插入 BST，画出这样构造出的 BST 树，并写出对该 BST 按照前序遍历得到的序列。

解答：

插入 {18, 73, 10, 5, 68, 99, 27, 41, 51, 32, 25} 得到的 BST 边关系为：18 的左、右孩子为 10, 73；10 的左孩子为 5；73 的左、右孩子为 68, 99；68 的左孩子为 27；27 的左、右孩子为 25, 41；41 的左、右孩子为 32, 51。前序遍历为 18, 10, 5, 73, 68, 27, 25, 41, 32, 51, 99。

14. (6 分) 以下是计算模式 P 的 next 向量的算法：

```
def find_next(P):
    i, k = 0, -1
    m = len(P)
    nxt = [0] * m
    nxt[0] = -1
    while i < m:
        while k >= 0 and P[i] != P[k]:
            k = nxt[k]
        i += 1
        k += 1
        if i == m:
            break
        if P[i] == P[k]:
            nxt[i] = nxt[k]
        else:
            nxt[i] = k
    return nxt
```

采用上述算法，计算模式 P="abcdaabcbab" 的 next 向量。

i
P[i]
a
b

c
d
a
a
b
c
a
b
next[i]

解答：

对 $P = \text{abcdaabcb}$ ，按题中优化 next 算法得：

i	0	1	2	3	4	5	6	7	8	9
$P[i]$	a	b	c	d	a	a	b	c	a	b
$\text{next}[i]$	-1	0	0	0	-1	1	0	0	3	0

15. (6 分) 数组 $\{23, 17, 14, 6, 13, 10, 1, 5, 8, 12\}$ 是否为一个最大值堆？若是，请说明理由，否则请严格按照筛选法建堆的过程将其调整成为最大值堆，并画出逐步调整建堆的过程。

解答：

原数组不是最大值堆，因为根 23 的子结点 17, 14 虽小于根，但结点 17 的子树中有 13、14 等局部关系需按筛选法统一检查。自最后一个非叶结点向前筛选，最终最大值堆可为层序 $\{23, 17, 14, 8, 13, 10, 1, 5, 6, 12\}$ ；若严格使用常规下滤，关键交换为 $6 \leftrightarrow 8$ ，其余父子关系满足最大堆要求。

16. (6 分) 对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵 (多的那个作为新树根，少的那个根作为新根的直接子结点)；若两棵树规模同样大，则把根值较大的并入根值较小 (新树根取值小的)。同时，采用路径压缩优化。(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10) (8,7) (7,0) (10,15) (10,13)

解答：

按重量权衡合并并在查找时路径压缩，所有 $0 \sim 15$ 最终属于同一个等价类，代表元为 0。合并时把规模较小的树挂到规模较大的树根下，规模相同时按题给规则选择代表元；每次 find 会把访问路径上的结点直接连到根。若题目要求“最终完全压缩”的树，可画成以 0 为根、其余结点均直接指向 0 的星形等价类树；若只压缩实际执行过 find 的路径，则中间若干结点可能仍保留二级父指针，但集合划分不变。

三、算法填空 (每空 2 分，共 16 分)

17. 1. 下面的算法将一个用带度数的后根次序法表示的森林转换为左子结点/右兄弟结点法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。空格中可能需要填写 0 到多条语句 (或表达式)。

```
class Stack:
    def empty(self): ...
    def size(self): ...
    def top(self): ...
    def push(self, x): ...
    def pop(self): ...

class Node:
    def __init__(self, info, degree):
```

```

        self.info = info
        self.degree = degree

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.child = None
        self.sibling = None

    def set_child(self, pointer):
        self.child = pointer

    def set_sibling(self, pointer):
        self.sibling = pointer

def convert(nodes):
    cstack = Stack()
    for i in range(len(nodes)):
        cur = TreeNode(nodes[i].info)
        if nodes[i].degree == 0:
            # 填空 1
        else:
            assert nodes[i].degree <= cstack.size()
            temp2 = None
            for j in range(____):      # 填空 2
                temp1 = cstack.top()
                cstack.pop()
                # 填空 3
                temp2 = temp1
            # 填空 4
            cstack.push(cur)

    cur = temp2 = None
    while not cstack.empty():
        cur = cstack.top()
        cstack.pop()
        # 填空 5
        temp2 = cur
    return cur

```

解答：

带度数后根序转孩子兄弟表示的填空：

- (1) `cstack.push(cur);`
- (2) `nodes[i].degree;`
- (3) `temp1.set_sibling(temp2);`
- (4) `cur.set_child(temp2);`
- (5) `cur.set_sibling(temp2)。`

其思想是：后根序中某结点出现时，它的所有子树根已经在栈顶，依次弹出并用右兄弟指针串接，再把最左孩子接到当前结点。

18. 2. 给定一个二叉树的根结点和树中任意两个结点，补全下面的程序段以求这两个结点的最近公共祖先。

```

def lowest_common_ancestor(root, p, q):
    if ____:      # 填空 6
        return root

```



```

left = lowest_common_ancestor(root.left, p, q)
right = lowest_common_ancestor(root.right, p, q)
if ____: # 填空 7
    return root
return left if ____ else right # 填空 8

```

解答:

最近公共祖先填空:

- (1) `root is None or root == p or root == q;`
- (2) `left is not None and right is not None;`
- (3) `left is not None.`

递归含义是: 在以 `root` 为根的子树中查找 p, q 的最近公共祖先, 若当前根为空或已经遇到 p/q , 就把当前根返回给上一层。分别递归左右子树后, 若左右返回值都非空, 说明 p, q 分别位于当前根两侧, 当前根就是最近公共祖先; 若只有左边非空, 则答案在左子树; 若只有右边非空, 则答案在右子树。每个结点最多访问一次, 时间复杂度 $O(n)$ 。

四、算法设计与实现 (16 分)

19. (8 分) 现有两个满足先进先出原则的队列 A 和 B , 在队列上可进行以下 4 个操作: `front()`: 获得队首元素; `push_back(T x)`: 把元素 x 插到队尾; `pop_front()`: 删除队首元素; `empty()`: 判断是否为空。请用 A 和 B 模拟一个栈的 4 个操作: `top()`, `pop()`, `push()`, `empty()`。只要给出实现栈操作的基本思想, 对时间效率无具体要求。

解答:

两个队列模拟栈: 令 A 保存当前栈元素且队首为栈顶。`push(x)` 时先把 x 入空队列 B , 再把 A 中所有元素依次出队并入队到 B , 最后交换 A, B ; `top()` 返回 A 的队首; `pop()` 删除并返回 A 的队首; `empty()` 判断 A 是否为空。这样四个栈操作均正确, 其中 `push` 为 $O(n)$, 其余为 $O(1)$ 。

20. (8 分) 对于两个串 A 和 B , 给出一种算法使得能够保证正确地求出最大的 L 值, 使得 A 串中长为 L 的前缀与 B 串中长为 L 的后缀相等。

解答:

求最大的 L 使 A 的长度为 L 的前缀等于 B 的长度为 L 的后缀: 构造串 $S = A + \# + B$, 其中 $\#$ 不在两个串中出现; 计算 KMP 前缀函数, 最后一个位置的前缀函数值即为答案。时间复杂度 $O(|A| + |B|)$ 。

五、分析证明题 (共 16 分)

21. (6 分) 试证明在一棵树中, u 是 v 的祖先, 当且仅当在先序遍历中 u 在 v 之前且在后序遍历中 u 在 v 之后。

解答:

若 u 是 v 的祖先, 先序遍历先访问祖先再访问其子树, 故 u 在 v 前; 后序遍历先访问子树再访问祖先, 故 u 在 v 后。反之, 若先序中 u 在 v 前且后序中 u 在 v 后, 则 v 必落在 u 的子树访问区间内, 否则两个结点所属的不相交子树在先序和后序中的相对次序会相同, 矛盾。因此 u 是 v 的祖先。

22. (10 分) 对于满二叉树:

- (1) 试问具有 n 个叶结点的树中共有多少个结点?
- (2) 试证明:

, 其中 n 为叶结点的个数, il 表示第 i 个叶结点所在的层次 (设根结点所在的层次为 0)。

解答：

满二叉树每个内结点度为 2。若叶结点数为 n ，内结点数为 $n - 1$ ，总结点数为 $2n - 1$ 。又因为每个叶结点深度为 l_i 时对应一个前缀码字，满二叉树的叶结点恰好构成完备前缀码，故 Kraft 等式成立：

$$\sum_{i=1}^n 2^{-l_i} = 1.$$

数据结构与算法 A

common-16 秋-期末

一、选择填空题 (每空 1 分, 共 11 分)

1. G 是一个非连通无向图, 共有 21 条边, 则图 G 至少有 8 个顶点。

解答:

答案为 8。要在顶点数尽量少的前提下容纳 21 条边且保持非连通, 应把尽可能多的顶点放在同一个完全图分量中, 另留至少一个孤立顶点。于是 v 个顶点的非连通简单无向图最多有 $\binom{v-1}{2}$ 条边。 $v = 7$ 时最多 $\binom{6}{2} = 15$ 条, 不够; $v = 8$ 时最多 $\binom{7}{2} = 21$ 条, 恰好可由 K_7 加一个孤立点实现, 所以至少 8 个顶点。

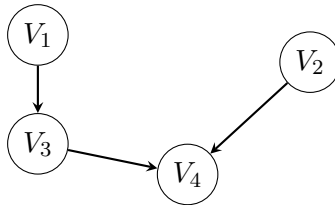
2. 对于一个包含 $N(N > 1)$ 个顶点的图, 假定任意两点间最多只有一条边, 那么下列哪些情况是错误的 _____。

- (A) 如果是有向图, 则其任何一个极大强连通子图都无法进行拓扑排序。
- (B) 如果是无向连通图, 则其最小生成树一定不包括权重最大的边。
- (C) 如果是无向连通图, 假设所有边的权重均为正值, Dijkstra 算法给出的生成树不一定是最小生成树, 但是与该图的任何一个最小生成树都至少有一条相同边。

解答:

错误项为 A、B。强连通分量若多于一个顶点, 则任意两点互相可达, 必然含有有向环, 不能作为 DAG 做拓扑排序; 但单点强连通分量本身没有环, 所以“强连通分量一定不能拓扑排序”的绝对说法错误。无向连通图的最小生成树也可能包含全图权重最大的边, 例如该边是连接某个割的唯一边时, 由割性质可知它必须被选入 MST, 因此“最大权边一定不在 MST 中”也错误。

3. 有向图 G 如下图所示:



- (1) 写出所有可能的拓扑序列: _____。
- (2) 若要使该图只有唯一的拓扑序列, 则可以添加一条弧 _____。

解答:

拓扑序列为 1234, 1324, 2134。枚举时先列出入度为 0 的顶点, 每次删除一个当前入度为 0 的顶点并更新后继入度; 若某一步有多个入度为 0 的顶点, 就产生不同拓扑序列。要使拓扑序列唯一, 需要消除关键分叉: 添加弧 $v_2 \rightarrow v_1$ 可强制 2 在 1 前, 唯一序列为 2134; 添加弧 $v_3 \rightarrow v_2$ 可强制 3 在 2 前, 唯一序列为 1324。所加弧不能造成环, 否则就不再存在拓扑序列。

4. 在快速排序中, 定义一次平分的划分为“幸运的划分”, 而一次划分如果有一边为空则是“不幸的划分”。假设划分的过程总是“幸运”和“不幸”交替的, 则该快速排序的时间复杂性为 $n \log n$ 。

解答:

时间复杂性为 $O(n \log n)$ 。一次“幸运划分”把问题分成大约相等的两半, 递归深度贡献为 $\log n$; 一次“不幸划分”虽然只去掉很少元素, 但题设说明幸运和不幸交替出现, 因此每两层递归后问题规模仍会

至少缩小为常数比例。每一层划分扫描当前子数组，所有同层子问题的扫描量合计为 $O(n)$ ，层数仍为 $O(\log n)$ ，所以总复杂度为 $O(n \log n)$ ，不是快速排序最坏情形的 $O(n^2)$ 。

5. 具有 10000 个关键码的 16 阶 B 树的查找路径长度 (从根到叶节点访问 B 树索引块的次数) 不会小于 3。

解答：

答案为 3。题目问“不会小于”多少，因此只需证明路径长度不可能是 1 或 2。16 阶 B 树每个结点最多 15 个关键码、16 个孩子。若查找路径长度按访问 B 树索引块层数计，两层最多容纳

$$15 + 16 \cdot 15 = 255$$

个关键码，远小于 10000，所以查找路径长度至少为 3。本题填空给的是下界，不要求继续判断 3 层是否一定足够；若改问精确高度，还需要结合根、非根结点的最小/最大关键码数重新估计。

6. 设有 8 个初始归并段，其长度分别为 32, 46, 56, 64, 20, 87, 70, 40；进行 3 路归并排序，所构造的最佳归并树对应的总读写次数为 1590。

解答：

最佳 3 路归并树的带权外部路径长度为 795，总读写次数为 $2 \times 795 = 1590$ 。3 路归并要求构造 3 叉 Huffman 树；因初始归并段数为 8，满足 $(n-1) \bmod (3-1) = 1$ ，需补一个长度为 0 的虚段，使叶子数变为 9。每次合并三个最短段并把新段重新放回候选集合，所有合并产生的新段长度之和就是 WPL；外排序每条记录每过一层通常读一次写一次，因此总 I/O 量为 $2WPL$ 。

7. $A[N][N]$ 是对称矩阵，现将下三角矩阵按行存储到一维数组 $T[N(N+1)/2]$ 中 (包括对角线)，则对任一上三角元素 $A[i][j]$ 其对应值 ($0 \leq i \leq j < N$) 在 $T[k]$ 中的下标 k 是 $j(j+1)/2+i$ 。

解答：

下标为 $k = j(j+1)/2 + i$ ，这里默认数组从 0 开始编号且题中访问的是上三角元素 $A[i][j] (i < j)$ 。对称矩阵满足 $A[i][j] = A[j][i]$ ，而下三角按行存储时，第 j 行之前共有 $0+1+\dots+j = j(j+1)/2$ 个元素，本行中 $A[j][i]$ 的偏移为 i ，故总下标为 $k = j(j+1)/2 + i$ 。若采用 1 下标，公式需相应减去常数偏移。

8. 在一棵空 AVL 树中，顺序插入如下关键码：{5, 9, 4, 2, 1, 3, 8}，请问全部插入后，在等概率下查找成功的平均检索长度为 $17/7$ 。

解答：

最终 AVL 树各结点查找长度和为 17，等概率成功平均检索长度为 $17/7$ 。插入序列 {5, 9, 4, 2, 1, 3, 8} 时，插入 1 后在结点 4 处出现 LL 型失衡，需右旋；插入 3 后在相应祖先处出现 LR 型调整，先左旋再右旋；插入 8 后保持平衡。按最终树从根开始把根的查找长度计为 1，逐层累计 7 个结点的查找长度之和为 17，所以平均成功查找长度为 $17/7$ 。

9. 已知广义表 $C = (c, (d, A), B, e)$ ，则广义表 C 的深度为 2， $\text{tail}(\text{head}(\text{tail}(C)))$ 的运算结果为 (A)。

解答：

广义表深度为 2； $tail(head(tail(C))) = (A)$ 。因为 $C = (c, (d, A), B, e)$ 的元素中只有第二个元素 (d, A) 是子表，且该子表内部不再出现更深的括号，所以最大括号嵌套层数为 2。计算表达式时， $tail(C) = ((d, A), B, e)$ ， $head(tail(C)) = (d, A)$ ，再取其尾部得到 (A) ；注意广义表的 $tail$ 返回的是“去掉表头后的子表”，不是单个原子 A 。

二、简答辨析题 (每题 3 分，共 15 分)

10. 如果要找出一个具有 n 个元素集合中的第 k ($1 \leq k \leq n$) 个最小元素，所学过的排序方法中哪种最适合？给出实现的基本思想。

解答：

最适合用快速选择思想。任选或随机选一个枢轴，把数组划分为“小于枢轴、等于枢轴、大于枢轴”三部分；若枢轴最终秩为 k ，它就是第 k 小元素；若枢轴秩大于 k ，只需递归处理左侧；若枢轴秩小于 k ，只需在右侧查找相应的新秩。与快速排序不同，快速选择每层只递归进入一边，平均递推为 $T(n) = T(n/2) + O(n)$ ，故平均时间为 $O(n)$ ，额外空间可做到 $O(1)$ 或递归栈 $O(\log n)$ 。

11. 已知一组关键码为 $(26, 36, 41, 38, 44, 15, 68, 12, 06, 51, 25)$ ，散列表长度为 15，用线性探查法解决冲突构造这组关键码的散列表。散列函数为 $h(k) = k \bmod 13$ 。请回答：
- (1) 构造顺序插入上述关键码集合后的散列表；
 - (2) 下一记录放到第 11 个槽和第 7 个槽中的概率分别是多少？
 - (3) 查找成功和失败情形下的平均查找长度 ASL 分别是多少？

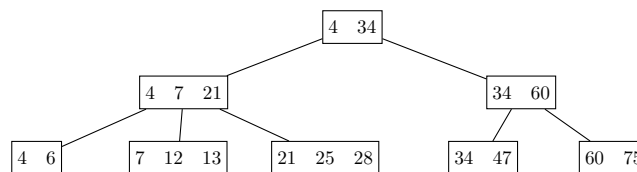
解答：

顺序插入后散列表为：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
关键码	26	25	41	15	68	44	6	-	-	-	36	-	38	12	51

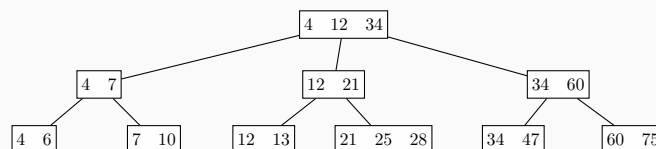
下一记录放到第 11 个槽的概率为 $2/13$ ，放到第 7 个槽的概率为 $9/13$ 。查找成功的平均查找长度为 $20/11$ ；查找失败的平均查找长度为 $52/13 = 4$ 。

12. 有如下图所示的一个 3 阶 B+ 树，请分别画出依次插入关键码 10 和删除关键码 4 的 B+ 树，并分析上述操作过程中的访外读写次数。



解答：

插入 10 后根结点变为 $[4, 12, 34]$ ，对应叶结点分裂为 $[4, 6]$ 、 $[7, 10]$ 、 $[12, 13]$ 、 $[21, 25, 28]$ 、 $[34, 47]$ 、 $[60, 75]$ ，读 3 次、写 5 次：



删除 4 后根结点为 $[6, 34]$ ，左内部结点为 $[6, 12, 21]$ ，右内部结点为 $[34, 60]$ ，写 3 次：

15. 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下：

```
class Graph:
    def first_edge(self, vertex): ...
    def next_edge(self, edge): ...
    def is_edge(self, edge): ...
    def to_vertex(self, edge): ...

visited = [False] * MAXSIZE

def get_path_num_len(G, i, j, length):
    if i == j and length == 0:
        return 1

    total = 0
    visited[i] = True
    e = G.first_edge(i)
    while G.is_edge(e):
        v = G.to_vertex(e)
        if not visited[v]:
            total += get_path_num_len(G, v, j, length - 1)
        e = G.next_edge(e)

    visited[i] = False
    return total
```

解答：

填空为 $i == j$ and $length == 0$ ；递归累加语句为 $total += get_path_num_len(G, v, j, length - 1)$ 。递归的出口表示“已经走完规定长度”，只有此时当前位置正好是终点 j 才返回 1。否则枚举当前顶点 i 的每个未访问邻接点 v ，把问题化为从 v 到 j 、剩余长度为 $length - 1$ 的路径条数。为保证路径简单，进入递归前后要正确设置和恢复访问标记。

16. 下面的代码实现了一种计数排序：对每个待排序记录，扫描整个序列统计比它小的记录个数 $count$ ， $count$ 即是该记录在序列中的争取位置。请将下面是计数排序的程序代码补充完整。

```
def sort_records(array):
    cur_index = 0          # 待排下标
    n = len(array)
    while cur_index < n:
        count = 0
        for j in range(n):    # 统计比 array[cur_index] 小的值的个数
            if array[j] < array[cur_index]:
                count += 1

        array[cur_index], array[count] = array[count], array[cur_index]
        if cur_index == count:
            cur_index += 1
    return array
```

解答：

填空为统计所有小于当前记录的元素个数 $count$ ，将当前记录交换到 $count$ 位置；若当前位置已经正确则推进 $curIndex$ 。该算法需注意重复关键码时要用稳定计数或额外规则处理。

四、设计分析题 (共 9 分)

17. (3 分) 某小区有 N 座别墅需要供水。在第 i 座别墅里挖井需要 $w[i]$ 的费用，在第 i 座和第 j 座别墅之间铺水管需要 $c[i][j]$ 的费用。给每座别墅供水，要么挖井、要么跟其他有井的别墅铺设连通的水管路径。请设计算法，求解使每座别墅都得到供水的最小费用方案。

解答：

构造一个额外水源顶点 s 。从 s 到第 i 座别墅连一条边，权重为 $w[i]$ ，表示“在第 i 座别墅挖井”；别墅 i, j 之间连边权重为 $c[i][j]$ ，表示铺设两者之间的管道。这样，任何让所有别墅有水的方案都对应于扩展图中连接 s 和全部别墅的一棵连通子图；去掉其中的环不会增加费用，所以最优方案一定是一棵生成树。对该图求最小生成树，选中的 $s-i$ 边表示挖井，选中的别墅间边表示铺管，即为最小费用供水方案。

18. (6 分) 现有一个工资系统，请实现满足如下操作需求的 Splay 树，并给出算法的伪代码：(Splay 树根为 $root$ ，其他变量可以自己定义)

1) $insert(w)$ ：新加一位工资为 w 的员工 (假设 w 没有重复值)；

2) $fire(t)$ ：解雇工资大于 t 的员工。

相关定义和函数说明如下：

伸展树是一种自平衡的 BST，数据结构如下：

```
class TreeNode:
    def __init__(self, wage):
        self.wage = wage
        self.father = None
        self.left = None
        self.right = None
```

可以直接使用的函数：

```
def Splay(x, f):
    """ 将 x 旋转为 f 的子结点；f 为 None 时把 x 旋到根。"""

def find(x, subtree):
    """ 在 subtree 中查找 x；若失败，返回 x 应插入位置的父结点。"""

def Delete(x):
    """ 删除以 x 为根的子树。"""

root = None

def insert(w):
    global root
    if root is None:
        root = TreeNode(w)
        return

    parent = find(w, root)
    new_node = TreeNode(w)
    new_node.father = parent
    if w < parent.wage:
        parent.left = new_node
```



```
    else:
        parent.right = new_node
    Splay(new_node, None)
    root = new_node

def fire(t):
    global root
    if root is None:
        return

    boundary = find(t, root)
    if boundary.wage == t:
        Splay(boundary, None)
        Delete(boundary.right)
        boundary.right = None
        root = boundary
        return

    parent = boundary
    sentinel = TreeNode(t)
    sentinel.father = parent
    if t < parent.wage:
        parent.left = sentinel
    else:
        parent.right = sentinel

    Splay(sentinel, None)
    Delete(sentinel.right)
    root = sentinel.left
    if root is not None:
        root.father = None
```

解答：

插入时按 BST 规则定位工资 w 的插入父结点，挂接新结点后将新结点 splay 到根。解雇工资大于 t 的员工时，先把分界工资 t 或其应插入位置伸展到根，再删除根右侧所有大于 t 的子树；若临时插入了分界结点，最后删除该临时结点并恢复左右子树。

数据结构与算法 A

common-17 秋-期中

一、选择与填空 (每空 2 分, 共 24 分)

1. 下面函数的时间复杂度是 ()

```
def recursive(n, m, k):
    if n <= 0:
        print(m, k)
    else:
        recursive(n - 1, m + 1, k)
        recursive(n - 1, m, k + 1)
```

- (A) $O(n * m * k)$ (C) $O(2^n)$
(B) $O(n^2 * m^2)$ (D) $O(n!)$

解答:

选 $O(2^n)$ 。该递归每次把规模为 n 的问题分成两个规模为 $n - 1$ 的子问题, 除递归调用外只做常数或低阶工作, 因此可写成 $T(n) = 2T(n - 1) + O(1)$ 。展开递归树, 第 i 层有 2^i 个结点, 直到深度 n , 结点总数为 $1 + 2 + \dots + 2^n = O(2^n)$ 。所以指数项主导, 不能化简成多项式复杂度。

2. 完成在双循环链表结点 p 之后插入 s 的操作为 ():

- (A) $p.next.prev=s;$ $s.prev=p;$ $s.next=p.next;$ (C) $s.next=p.next;$ $s.prev=p;$ $p.next.prev=s;$
 $p.next=s;$ $p.next=s;$ $s.prev=p;$ (D) $s.prev=p;$ $s.next=p.next;$ $s.prev.next=s;$
 $s.next=p.next;$ $s.next.prev=s;$

解答:

A、C、D 的赋值次序均能正确完成在双循环链表结点 p 后插入 s 。正确插入需要同时维护四条链接: s 的前驱为 p , s 的后继为原来的 $p.next$, 原后继的前驱改为 s , p 的后继改为 s 。B 错在过早令 $p.next=s$, 随后 $s.next=p.next$ 会读到已经被改写的新值, 使 $s.next$ 指向自身, 原来的后继结点丢失。

3. 设栈 S 和队列 Q 初始状态为空, 元素 $e_1, e_2, e_3, e_4, e_5, e_6$ 依次通过栈 S , 一个元素出栈后即进队列 Q , 若 6 个元素出队序列是 $e_2, e_4, e_3, e_6, e_5, e_1$, 则栈 S 的容量至少是 ()

- (A) 2; (C) 4;
(B) 3; (D) 6

解答:

容量至少为 3。过程为: 压入 e_1, e_2 后弹出 e_2 ; 压入 e_3, e_4 后弹出 e_4, e_3 ; 压入 e_5, e_6 后弹出 e_6, e_5, e_1 , 最大栈深为 3。

4. 设循环队列的容量为 40 (序号从 0 到 39), 现经过一系列的入队和出队运算后: 1) $front=12, rear=19$; 2) $front=19, rear=12$; 在这两种情况下, 循环队列中的元素个数分别为 _____ 和 _____。

解答：

按通常约定 `front` 指向队首、`rear` 指向下一个可插入位置，循环队列元素个数为

$$(rear - front + 40) \bmod 40.$$

把题给两组 `front`、`rear` 代入即可分别得到 7 和 33。公式中加上容量 40 是为了避免 `rear < front` 时出现负数，最后再取模回到合法范围。

5. 一个 n 位的字符串共有 _____ 个子串。

解答：

长度为 n 的字符串中，长度为 1 的子串有 n 个，长度为 2 的有 $n-1$ 个，依此类推，长度为 n 的有 1 个。因此非空子串总数为 $n + (n-1) + \cdots + 1 = n(n+1)/2$ ；若教材把空串也计作子串，则再加 1。

6. 已知二叉树有 n 个结点 ($n > 0$)，则度为 2 的结点最多有 _____ 个。

解答：

度为 2 的结点最多为 $\lfloor (n-1)/2 \rfloor$ 。设度为 0, 1, 2 的结点数分别为 n_0, n_1, n_2 ，二叉树恒有 $n_0 = n_2 + 1$ 。总结点数 $n = n_0 + n_1 + n_2 = 2n_2 + n_1 + 1$ ，因此 $n_2 = (n - n_1 - 1)/2$ 。要让 n_2 最大，应使度为 1 的结点数 n_1 尽量小：当 n 为奇数取 $n_1 = 0$ ，当 n 为偶数取 $n_1 = 1$ ，统一得到 $\lfloor (n-1)/2 \rfloor$ 。

7. 用数组存储一个有 50 个元素的最小值堆。已知堆中没有重复元素。给出最大元素的下标可能的取值范围为 _____ 到 _____。

解答：

若堆数组采用 0 下标，最大元素只能出现在叶结点位置，范围为 25 到 49；若采用 1 下标，则为 26 到 50。因为这是小根堆，父结点关键字不大于孩子，所以最大关键字不可能出现在任何内部结点，否则它的孩子会更大或等大而违反“最大”的位置判断。含 50 个元素的完全二叉树中，0 下标最后一个内部结点为 $\lfloor (50-2)/2 \rfloor = 24$ ，叶子从 25 开始；1 下标最后一个内部结点为 $\lfloor 50/2 \rfloor = 25$ ，叶子从 26 开始。

8. 假设用于通信的电文由 5 个字符组成。已知其中一个字符的 Huffman 编码为 1101，则该 Huffman 编码树的平均编码长度为 _____。

解答：

仅给出一个 Huffman 码字 1101 不能唯一确定按频率加权的平均编码长度。平均编码长度需要知道每个字符的频率以及所有字符的码长；一个码字只能说明其中某个叶子的深度为 4，不能推出整棵 Huffman 树。若补充约定为 5 个字符等概率，且码长集合为 1, 2, 3, 4, 4，则平均编码长度为

$$\frac{1 + 2 + 3 + 4 + 4}{5} = \frac{14}{5}.$$

若权值不同，即使码长集合相同，加权平均值也会改变。

9. 假设先根次序遍历某棵树的结点次序为 GABCDIJEFH，
后根次序遍历该树的结点次序为 BADIJC FEHG。那么 I 结点的父结点是 _____。

解答：

I 的父结点是 C。先根序第一个结点 G 为根，后根序最后一个结点也为 G；递归分割可知 G 的孩子依次为 A, C, E, H。在 C 对应的子序列中，先根片段为 C D I J，后根片段为 D I J C，所以 C 的孩子为

D, I, J , 从而 I 的父结点为 C 。

10. 一个拥有 144 个结点的完全三叉树, 现在从倒数第二层上任取一个结点, 该结点为叶结点的概率为_____。

解答:

补全题意“从倒数第二层任取一个结点, 该结点为叶结点的概率”。完全三叉树前 5 层共有 $1 + 3 + 9 + 27 + 81 = 121$ 个结点, 144 个结点说明第 6 层有 23 个结点, 占据第 5 层前 $\lceil 23/3 \rceil = 8$ 个父结点, 因此第 5 层叶结点数为 $81 - 8 = 73$, 概率为 $73/81$ 。

二、辨析与简答 (共 24 分)

11. (6 分) 现有一个由单链表和循环单链表构成的特殊链表, 单链表的头元素是 head, 末尾元素为 tail, head...tail 即是一条普通的链表, 同时 tail 元素还是另一条循环单链表的头元素。如 A.B.C.D.E.F.G.H.E 就是一个特殊链表, 其中 head 为 A, tail 为 E。
现在你只知道 head, 但是你知道这个循环链表中有多少元素, 但同时你的剩余空间十分的小, 所以你想用尽可能小的空间来解决问题, 请问你会怎么做? (依据占用空间给分)

解答:

用 Floyd 快慢指针。先令慢指针每次走一步、快指针每次走两步, 若链表有环, 两者一定会在环内相遇; 若快指针遇到空指针, 则不是循环链表。相遇后固定其中一个指针, 从相遇点沿 next 继续走, 直到再次回到相遇点, 走过的边数就是环长; 对于纯循环链表, 这个环长也就是元素个数。算法只使用常数个指针和计数器, 空间复杂度 $O(1)$, 时间复杂度 $O(n)$ 。

12. (6 分) 假设以 I 和 O 分别表示入栈和出栈操作, 栈的初始和终态均为空, 入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列, 称可以操作的序列为合法序列, 否则为非法序列。如 IOIOIOIO 合法, IOOIOIO 和 IIOIOIO 为非法。请给出一个算法, 判断所给操作序列是否合法。

解答:

扫描操作串, 用计数器记录当前栈深: 遇到 I 加 1, 遇到 O 减 1。任意前缀中 O 的数量不得超过 I, 即栈深不能为负, 否则说明在空栈上执行了出栈操作; 扫描结束时 I 与 O 数量相等, 即栈深为 0, 否则说明仍有元素未弹出或出栈次数过多。两条件同时满足则该 I/O 序列合法。若题目还给定栈容量 m , 则扫描过程中还需额外检查栈深是否超过 m 。

13. (6 分) 给定一棵用数组存储的完全二叉树 {10, 5, 12, 3, 2, 1, 8}
- (1) 用筛选建堆法将该完全二叉树调整为最大值堆。画出调整后得到的最大值堆, 并说明在调整过程中都依次交换了哪些元素。(注: 画堆只需要写出层次遍历序列结果即可)
 - (2) 将 6、7、14 按顺序插入 (1) 得到的最大值堆, 堆的空间足够大, 画出插入 14 后得到的最大值堆, 并说明在插入 14 的过程中都依次交换了哪些元素。
 - (3) 对 (2) 得到的堆进行 3 次 deleteMax, 画出第 3 次 deleteMax 后得到的堆, 并说明在第 3 次 deleteMax 的过程中都依次交换了哪些元素。

解答:

初始数组 {10, 5, 12, 3, 2, 1, 8} 经筛选建最大堆得 {12, 5, 10, 3, 2, 1, 8}, 主要交换 $10 \leftrightarrow 12$ 。依次插入 6, 7, 14 后, 插入 14 的交换为 $14 \leftrightarrow 2$ 、 $14 \leftrightarrow 7$ 、 $14 \leftrightarrow 12$, 得到 {14, 12, 10, 6, 7, 1, 8, 3, 5, 2}。随后连续 3 次 deleteMax 后堆为 {8, 7, 5, 6, 2, 1, 3}; 第 3 次删除中的下滤交换为 $3 \leftrightarrow 8$ 、 $3 \leftrightarrow 5$ 。

14. (6 分) 对下列 15 个等价对进行合并, 给出所得等价类树的图示。在初始情况下, 集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则, 合并时子树结点少的并入结点数多的那棵 (多的那个作为新树根, 少的那个根作为新根的直接子结点); 若两棵树规模同样大, 则把根值较大的并入根值较小 (新树根取值小的)。同时, 采用路径压缩优化。
(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (12,9) (4,14) (6,7) (8,10) (8,7) (7,11) (10,15) (10,13)

解答:

按题给重量权衡规则并在查找时路径压缩, 所有 $0 \sim 15$ 最终合并为一个等价类, 代表元可取 0。重量权衡保证小树挂到大树下, 避免树高快速增长; 路径压缩则在 `find` 返回时把访问路径上的结点直接改挂到根。若题目要求画“全部操作结束并完全压缩”的等价类树, 可画成以 0 为根, 其余结点均为 0 的孩子; 若只画严格按操作序列产生的中间形态, 应保留那些没有被最后一次 `find` 访问到的父子边。

三、算法填空 (每空 3 分, 共 24 分)

15. 下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 $1, 2, \dots, n$ 这 n 个连续的正整数组成。请补全下面的代码段, 使其可以判断给定的序列是否可以利用一个栈进行排序, 如果可以, 输出相应的操作。

例如: 序列 4 3 1 2, 此序列可以排序, 输出: push push push pop push pop pop pop。

```
class Stack:
    def clear(self): ...
    def push(self, item): ...
    def pop(self): ...
    def top(self): ...
    def is_empty(self): ...
    def is_full(self): ...

s = Stack()
sequence = [...]
n = len(sequence)

def is_legal():
    # 判断序列是否可以排序
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if ____: # 填空 1
                    return False
    return True

def transform():
    # 输出转换操作
    cur, i = 1, 0
    while ____: # 填空 2
        while s.is_empty() or ____: # 填空 3
            ____ # 填空 4
            print("push", end=" ")
            if cur == s.top():
                break
            s.pop()
            print("pop", end=" ")
            cur += 1
```

解答:

一个排列可由一个栈排成升序, 当且仅当不存在 $i < j < k$ 且 $sequence[k] < sequence[i] < sequence[j]$ 的 231 模式。填空为:

- (1) `sequence[k] < sequence[i] < sequence[j]`;
- (2) `cur <= n`;

```
(3) cur < s.top();
(4) s.push(sequence[i]); i += 1。
```

16. 下面的算法将一个用带右链的先根次序法表示的森林转换为用带度数的后根次序法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。

```
class RlinkTreeNode:
    def __init__(self, info, ltag, rlink=None):
        self.info = info
        self.ltag = ltag      # 1 表示没有左子结点
        self.rlink = rlink    # 右兄弟

class PostTreeNode:
    def __init__(self):
        self.info = None
        self.degree = 0

post_tree = [PostTreeNode() for _ in range(n)]
count = 0

def convert(node):
    """ 返回右兄弟链上的结点数量 (包括自己)。"""
    global count
    if node.ltag == 1:
        # 填空 5
        post_tree[count].degree = 0
        count += 1
    else:
        tmp = convert(next_node(node)) # 访问第一个子结点
        post_tree[count].info = node.info
        # 填空 6
        count += 1

    if ____: # 填空 7
        return convert(node.rlink) + 1
    else:
        # 填空 8
```

解答：

带右链先根序转带度数后根序的填空：

```
(1) post_tree[count].info = node.info;
(2) post_tree[count].degree = tmp;
(3) node.rlink is not None;
(4) return 1。
```

算法思想是先递归处理当前结点的所有孩子，再把当前结点写入后根序数组。变量 `tmp` 统计当前结点实际拥有的孩子个数，因此写入结点信息后还要写入度数。右链 `rlink` 表示兄弟关系，若兄弟存在，就继续沿右链处理并把兄弟也计入父结点的孩子序列；返回值用于告诉上一层当前子树贡献了一个孩子。这样输出顺序满足“先孩子、后根”，并且每个结点附带正确度数。

四、算法设计与实现 (14 分)

17. (8 分) 编写算法伪码，对给定的字符串 `str`，返回其最长重复子串及其下标位置。如 `str="abcdaccdac"`，

则子串"cdac" 是 str 的最长重复子串，下标为 2。
(重复子串允许重叠)

解答：

可用后缀数组。构造所有后缀及其起始下标，按字典序排序；最长重复子串一定是相邻后缀的最长公共前缀之一。依次计算相邻后缀 LCP，取最长者及较小起始下标即可。朴素实现时间 $O(n^2 \log n)$ ，若用线性 LCP 算法可降为 $O(n \log n)$ 或更优。

18. (6 分) 给出一个算法，求一棵树的树高。可以写出伪代码，需要相应的注释，或者写出详细算法思路。对时间复杂度无具体要求。

解答：

树高递归算法：若结点为空，返回 -1；若为叶子，返回 0；否则递归求所有孩子子树的高度，取最大值后加 1 作为当前结点高度。这个定义把高度理解为从该结点到最深叶子的边数，因此叶子高度为 0。若教材按“层数”定义树高，则空树为 0、叶结点为 1，相应地把上述返回值整体加 1 即可。遍历每个结点一次，时间复杂度 $O(n)$ 。

五、分析证明题 (共 14 分)

19. (8 分) 二叉树的内部路径长度是指所有节点的深度的和。假设将 N 个互不相同的随机元素插入一棵空的二叉搜索树。请证明得到的二叉搜索树的内部路径长度期望为 $O(N \log N)$ 。

解答：

随机插入得到的 BST 与快速排序递归树同分布。设 $E[I_N]$ 为内部路径长度期望，根的左子树规模 i 在 $0 \sim N-1$ 上等可能，因此

$$E[I_N] = N - 1 + \frac{1}{N} \sum_{i=0}^{N-1} (E[I_i] + E[I_{N-1-i}]),$$

解得 $E[I_N] = O(N \log N)$ 。

20. (6 分) 证明按先根次序周游树获得的序列与其对应二叉树的前序周游获得的序列相同。

解答：

树的孩子兄弟表示中，树的“第一个孩子”成为对应二叉树的左孩子，“右兄弟”成为右孩子。树的先根遍历先访问根，再依次访问各子树；对应二叉树的前序遍历也是先访问根，再沿左孩子进入第一个子树，并沿右孩子依次访问兄弟子树。因此两者访问序列相同。

数据结构与算法 A

common-17 秋-期末

一、选择填空题 (7 小题, 每空 1 分, 共 10 分)

1. 下列说法中正确的是 _____。

- (A) 图的广度优先搜索中邻接点的寻找具有”先进后出”的特征。
- (B) 连通图的生成树是该图的一个极大连通子图。
- (C) 图的广度优先搜索是一个递归过程。
- (D) 在非连通图的遍历过程中, 每调用一次深度优先搜索算法都得到该图的一个连通分量。

解答:

选 D。BFS 使用队列, 特点是先进先出, 而”先进后出”是栈的性质; 连通图的生成树包含全部顶点和 $n-1$ 条边, 是保持连通的极小连通子图, 不是极大连通子图; BFS 通常用队列迭代实现, 不是典型递归过程。非连通无向图中, 每次从一个尚未访问的顶点启动 DFS, 恰好访问到该顶点所在的连通分量, 因此 DFS 的启动次数等于连通分量数, D 正确。

2. 基于比较的内排序算法中, 平均时间复杂度为 $O(n \log n)$ 的算法有 _____, 这其中稳定的排序算法有 _____。

解答:

平均时间复杂度为 $O(n \log n)$ 的典型比较排序有快速排序、归并排序、堆排序。快速排序平均每层划分总工作量为 $O(n)$, 平均层数为 $O(\log n)$; 归并排序每层合并 $O(n)$ 、层数 $O(\log n)$; 堆排序建堆后做 n 次删除最大/最小, 每次 $O(\log n)$ 。其中稳定的是归并排序, 因为合并时可让相等元素按原相对顺序输出。

3. 在包含 n 个关键码的线性表中从后向前进行顺序检索 (0 下标为监视哨)。若第 i 个关键码的检索概率为 p_i , 且满足如下分布: $p_1 = 1/2, p_2 = 1/4, \dots, p_n = 1/2^n$, 则成功检索的平均检索长度为 _____, 失败检索的平均检索长度为 _____。

解答:

若题意为 $p_i = 2^{-i}$ 并按其归一化处理, 则

$$ASL_{succ} = \frac{\sum_{i=1}^n (n-i+1)2^{-i}}{1-2^{-n}}.$$

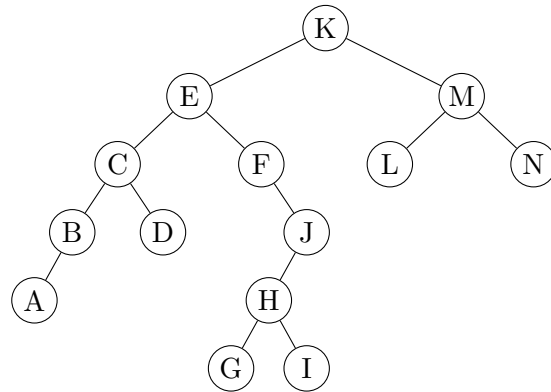
从后向前顺序检索失败时需检查全部 n 个关键码; 若把 0 下标监视哨也计为一次比较, 则 $ASL_{fail} = n+1$, 否则为 n 。

4. 给定一组输入关键码 {15, 1, 3, 14, 11, 2, 12, 10, 4, 17}, 采用大小为 3 的最小堆进行置换选择排序的输出为 _____。

解答:

用大小为 3 的最小堆。初始装入 15, 1, 3, 输出 1 后读入 14, 输出 3 后读入 11, 输出 11 后读入 2, 因 $2 < 11$ 被冻结到下一段; 继续输出 14, 15, 第一段为 1, 3, 11, 14, 15。被冻结和后续读入元素形成第二段, 排序后为 2, 4, 10, 12, 17。

5. 按照 AVL 定义, 下图所示的 BST 中没有达到平衡状态的结点有 _____。



解答：

不平衡结点为 K, F, J 。判断时从叶子向上计算每个结点左右子树高度，并检查平衡因子 $BF = h_L - h_R$ 是否属于 $\{-1, 0, 1\}$ 。结点 K 的左右子树高度差超过 1；结点 F 的右侧分支明显更深；结点 J 的左侧子树更深且差值超过 1。其余结点左右高度差不超过 1，所以仍满足 AVL 平衡条件。画图作答时应在每个被判不平衡的结点旁标出左右高度，避免只凭肉眼倾斜判断。

6. 假设磁盘块大小是 1024 字节，每个索引项需要 8 个字节。一个包含 20000 条记录的数据文件，若采用二级索引，则一级索引文件和二级索引文件共需占用 _____ 磁盘块。

解答：

每个磁盘块可放 $1024/8 = 128$ 个索引项。一级索引要为 20000 条记录各保存一个索引项，所以需要

$$\left\lceil \frac{20000}{128} \right\rceil = 157$$

块；二级索引再为这 157 个一级索引块建立索引，需要

$$\left\lceil \frac{157}{128} \right\rceil = 2$$

块。因此两级索引块总数为 $157 + 2 = 159$ 。这里计算的是索引占用块数，不包含原始数据文件块。

7. 给定两个广义表 $S = ((a, (b), c), ((d), e))$, $T = (f, (g, ((h)), i, j))$; 利用广义表的 Head 和 Tail 运算，则有 $d = \underline{\hspace{2cm}}$; $h = \underline{\hspace{2cm}}$ 。

解答：

$d = \text{Head}(\text{Head}(\text{Head}(\text{Tail}(S))))$; $h = \text{Head}(\text{Head}(\text{Head}(\text{Tail}(\text{Head}(\text{Tail}(T)))))$ 。广义表操作中，Head 取表的第一个元素，Tail 返回去掉第一个元素后剩余元素组成的表。求 d 时先通过 Tail(S) 去掉表头，再连续取表头进入嵌套子表，直到取得原子 d 。求 h 时同理，但需要先进入 T 的第二个元素对应的子表，再沿其第一分支继续取头。书写答案时要注意 Tail 的结果仍是表，因此经常需要再接一次 Head 才能进入目标子表。

二、简答辨析题 (5 小题，共 15 分)

8. (2 分) 将关键码序列 (7, 8, 30, 11, 18, 9, 14) 存储到一个散列表中。散列表是一个下标从 0 开始的一维数组，若散列函数采用: $H(\text{key}) = (\text{key} * 3) \text{ MOD } 7$ ，处理冲突采用线性探测法，装填 (负载) 因子要求为 0.7。(1) 请画出所构造的散列表; (2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

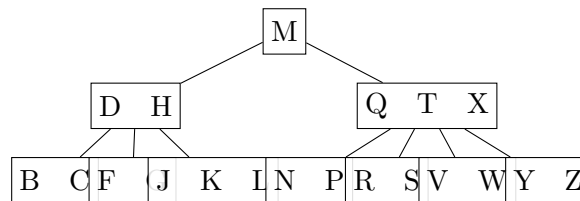
解答：

散列表长度满足 $7/L \leq 0.7$ ，故取 $L = 10$ 。依次插入 7, 8, 30, 11, 18, 9, 14, $H(\text{key}) = 3\text{key} \bmod 7$ ，线性探测得到：

地址	0	1	2	3	4	5	6	7	8	9
关键码	7	14	-	8	-	11	30	18	9	-

成功比较次数为 1,1,1,1,3,3,2, $ASL_{succ} = 12/7$ 。不成功查找从基地址 0 ~ 6 出发的比较次数为 3,2,1,2,1,5,4, $ASL_{fail} = 18/7$ 。

9. (4 分) 下图是一棵关键码为英文字符的 m 阶 B 树，依据字典序组织。请回答下述问题：



(1) 这是一棵合法的 B 树，则 m 可以取哪些值，并说明原因；(2) 若 m 取所有可能值中的最小值，则 (A) 给出查找字符 V 的过程，并说明需要的读盘次数 (假设根结点也需读盘操作)；(B) 给出插入字符 I ($H < I < J$) 的过程，并画出插入 I 后的 B 树；(C) 在原始 B 树 (即进行第 (B) 问的操作之前) 的基础上，给出删除 H 的过程，并画出删除 H 后的 B 树。

解答：

该树最大结点关键字数为 3，故 $m \geq 4$ ；又存在含 2 个关键字的非根结点，需满足 $\lceil m/2 \rceil - 1 \leq 2$ ，故 $m \leq 6$ 。所以 m 可取 4,5,6。取最小值 $m = 4$ 时，查找 V 的路径为根结点 [M]、结点 [Q,T,X]、叶结点 [V,W]，需读盘 3 次。插入 I 时定位到叶结点 [J,K,L]，插入后溢出并分裂，向父结点上推中间关键字；删除 H 时可用右侧子树中的后继关键字替换，再在对应叶结点删除后继关键字并检查是否下溢。

10. (4 分) 现有 8 个顺串，每个顺串的第一个关键码为 $S1[1..8] = \{17, 10, 2, 9, 13, 15, 12, 4\}$ ，第二个关键码是 $S2[1..8] = \{23, 34, 38, 25, 27, 40, 36, 39\}$ ，进行从小到大的归并。(1) 请写出初始构建的赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ (数组 A 和 B 存储的是 1 到 8 的顺串的下标， $B[0]$ 表示最终冠军)；(2) 在输出第一个全局赢者并重构赢者树或败者树后，写出此时赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ 。

解答：

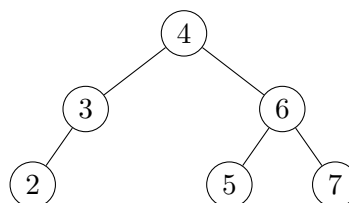
顺串归并题：第一轮全局最小来自第 3 个顺串，输出 2；随后第 8 个顺串首关键码 4 成为下一全局最小。赢者树/败者树数组应按“存顺串下标而非关键码”的约定填写，重构时只更新输出顺串所在路径。

11. (3 分) 请按照 2, 1, 4, 5, 9 的插入顺序，建立一棵红黑树，并画出每插入一个元素后的红黑树。(用 B 和 R 表示结点的颜色)

解答：

按 2,1,4,5,9 插入红黑树：插入 2 后根为黑；插入 1、4 为红，树仍平衡；插入 5 时父 4 与叔 1 同为红，变色为 1B,4B,2B,5R；插入 9 触发以 4 为祖父的 RR 调整，得到根 2B，左子 1B，右子 5B，5 的左子 4R、右子 9R。

12. (2 分) 给定下面的一棵 splay 树，按照结点序列 2, 3, 4, 5, 6, 7 访问所有结点之后，请画出访问后的 splay 树形结构。



解答:

按序访问并在每次访问后伸展到根，根依次变为 2,3,4,5,6,7。每次访问的关键码都比当前根大，因此访问路径主要沿右子树下降；伸展时根据父子和祖孙方向分别执行 zig、zig-zig 或 zig-zag，把被访问结点旋到根。连续按递增序列访问会把较大的结点不断旋到根，较小结点被保留在左侧。最终 7 为根，左孩子为 6，6 的左孩子为 5，依次形成左链 7-6-5-4-3-2；右子树为空。

三、算法填空题 (2 小题，每空 1 分，共 5 分)

13. 下面是考虑墓碑问题的散列表插入算法，算法中不允许有重复关键码插入。

```
class HashDict:
    EMPTY = object()
    TOMB = object()

    def __init__(self, size):
        self.M = size
        self.currnt = 0
        self.HT = [self.EMPTY] * size

    def p(self, key, i):
        """ 探查函数。"""

    def h(self, key):
        """ 散列函数。"""

    def get_key(self, elem):
        """ 获取元素 elem 的关键码。"""

    def eq(self, a, b):
        return a == b

    def hash_insert(self, elem):
        inplace = None
        i = 0
        home = self.h(self.get_key(elem))
        pos = home
        tomb_pos = False

        while ____:                # 填空 1
            if self.eq(elem, self.HT[pos]):
                return False
            if self.eq(self.TOMB, self.HT[pos]) and not tomb_pos:
                ____                # 填空 2
                tomb_pos = True
            i += 1
            ____                    # 填空 3: 探查

        if not tomb_pos:
            inplace = pos          # 若无墓碑，则插入第一个空槽
            self.HT[inplace] = elem
            self.currnt += 1
            return True
```

解答:

墓碑散列表插入:

(1) self.HT[pos] is not self.EMPTY and i < self.M;

```
(2) inplace = pos;
(3) pos = (home + self.p(self.get_key(elem), i)) % self.M。
```

带墓碑的开放定址插入要区分三类位置：真正空位 EMPTY、已删除但可复用的 TOMBSTONE、以及被有效记录占用的位置。探查时不能在墓碑处停止查找，因为后面可能已有同关键字记录；但应记下遇到的第一个墓碑作为候选插入位置。若最终遇到真正空位，则优先把新元素放到已记录的墓碑位置，否则放到该空位；第三空给出的探查公式用于按第 i 次探查继续计算下一个桶。循环结束后，若此前没有遇到墓碑，则插入到第一个空槽；若遇到墓碑，则复用第一个墓碑位置。

14. 设有一个仅由红、白、蓝 3 种颜色的条块组成的序列，各种色块的个数和分布是随机的，但 3 种颜色的总数为 n 。下面的代码实现了一种时间复杂度为 $O(n)$ 的排序，使得这些条块按照红、白、蓝的顺序排好。请将其补全。

```
RED, WHITE, BLUE = 1, 2, 3

def color_sort(array):
    insert_red, insert_white = 0, 0
    for i in range(len(array)):
        if array[i] == WHITE:
            array[i] = array[insert_white]
            array[insert_white] = WHITE
            insert_white += 1

        if array[i] == RED:
            ----- # 填空 4
            ----- # 填空 5
            array[insert_red] = RED
            insert_red += 1
            insert_white += 1
    return array
```

解答：

三色排序填空：

```
(1) array[i] = array[insert_white];
(2) array[insert_white] = array[insert_red]。
```

然后题中已有语句 `array[insert_red] = RED`，并同时推进两个插入边界。该算法维护三个区间边界：红色区在最前，白色区紧随其后，尚未扫描区和蓝色区在后。遇到红色元素时，需要把当前位置腾出，并把白区首元素后移到当前位置，再把红区尾后的元素移到白区首，最后在红区尾后写入 RED；因此两处填空正好完成这两次搬移。随后推进 `insert_red` 和 `insert_white`，即可保持“红、白、未处理、蓝”的循环不变式。

四、设计分析题 (2 小题，共 10 分)

15. (6 分) 请设计一个算法求有向无环图 (DAG) 中每对顶点间的“最长简单路径”(所谓“最长简单路径”，是指该简单路径包含的边数最多)。以一个有向无环图作为输入，对于每对顶点，如果它们之间存在简单路径，则输出其中路径长度最长 (边数最多) 的简单路径，否则输出空。试分析你所设计算法的时间复杂度。

解答：

对 DAG 先做拓扑排序。对每个源点 s ，按拓扑序进行动态规划： $\text{dist}[s]=0$ ，其余为 $-\infty$ ；扫描每条边 (u, v) ，若 $\text{dist}[u]+1 > \text{dist}[v]$ 则更新 $\text{dist}[v]$ 与前驱。对每个 s 做一次即可得到从 s 到所有点的最长简单路径。时间复杂度为 $O(|V|(|V| + |E|))$ ，若只求长度可直接保存距离，若要输出路径则同时保

存前驱并回溯。

16. (4 分) 利用证明基于比较的排序问题的复杂度下限的方法，试证明在已排序序列上基于比较的检索问题的复杂度下限是 $\Omega(\log n)$ 。

解答：

基于比较的有序表检索可看作决策树。若要判断目标属于哪个位置或哪个间隙，至少有 $n+1$ 个可能结果；一棵二叉比较决策树高为 h 时最多区分 2^h 个结果，故 $2^h \geq n+1$ ，即 $h \geq \lceil \log_2(n+1) \rceil = \Omega(\log n)$ 。